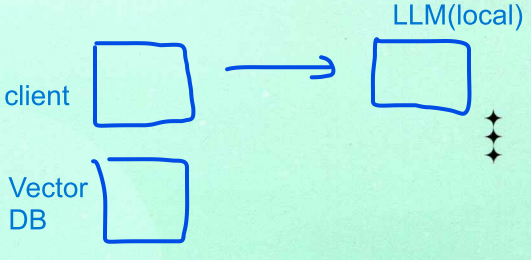
11장

로컬에서 딥시크-R1 모델 사용하기

GPT API를 이용하면 고품질 답변을 받을 수 있지만 비용이 발생하고 정보를 오픈AI로 보내서 처리하므로 보안 문제가 발생할 수 있습니다. 이런 문제를 해결하는 대안으로 로컬에서 실행할 수 있는 모델의 필요성이 크게 대두되었습니다. 이번 장에서는 GPT에 대항하는 모델로 떠오른 딥시크-R1DeepSeek-R1 모델을 랭체인에서 활용하는 방법을 다루겠습니다.

11-1 딥시크 모델 알아보기
11-2 랭체인에서 딥시크 모델 사용하기
11-3 딥시크에 기반한 RAG 만들기



11-1 딥시크 모델 알아보기

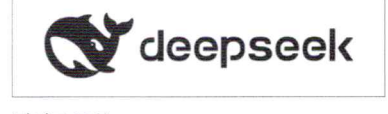
소규모 언어 모델의 등장

GPT가 API로 공개되었을 때 사람들은 인공지능 언어 모델을 자신이 개발하는 소프트웨어나 시스템에 손쉽게 탑재할 수 있자 열광했습니다. 하지만 GPT를 사용하려면 질의할 때마다 토큰 크기에 따른 비용이 발생하고 내 컴퓨터 안에 저장된 자료를 오픈AI의 서버에 전송해야 한다는 보안 문제가 있었습니다.

토큰당 부과되는 요금제는 언어 모델을 이용해 사용자에게 서비스를 제공하려는 기업이나 개인에게 골칫거리였습니다. GPT에 기반한 서비스를 개발해도 사용자가 지불하는 금액의 상당 부분은 오픈AI의 API 사용료로 지불해야 했기 때문입니다. 회사 내의 문서를 이용해 RAG를 개발하려는 경우에도 해당 문서를 오픈AI에 전송하게 되어 있어서 언어 모델을 사용할 수 없는 기업이 많았습니다.

이런 문제들을 해결하기 위해 일반 PC나 스마트폰에서도 실행할 수 있는 소규모 언어 모델Smal Language Model, SLM의 수요가 늘 있어 왔습니다. 이에 맞춰 대규모 언어 모델LLM을 저사양 PC에서도 작동할 수 있도록 성능을 다소 포기하고 경량화한 모델들이 등장했습니다.

딥시크-R1 모델



딥시크 로고

2024년 말 급부상한 중국 기업 딥시크는 자신의 회사명을 딴 딥시크 모델을 선보였습니다. 2025년 초 GPT의 성능에 필적할 만한 딥시크-R1DeepSeek-R1 모델을 누구나 사용할 수 있는 MIT 라이선스로 공개하며 전 세계에 충격을 안겼습니다. 딥시크는 '대형 언어 모델은 글로벌 IT 대기업이 아니면 개발할 수 없다'는 편견을 깨트렸습니다. 게다가 자신들이 개발한 여러 버전의 언어 모델을 오픈소스로 공개했습니다. 컴퓨터 사양이 충분하다면 딥시크-R1 모델을 자신의 컴퓨터에 내려받아 실행할 수 있습니다.

물론 일반 PC에서 대규모 언어 모델은 실행하기 어렵습니다. 딥시크 모델은 대규모 언어 모델을 경량화한 모델들도 함께 공개했습니다. 이 모델들도 기존의 소규모 언어 모델에서는 기대하기 어려웠던 수준의 답변을 내놓았습니다. 특히 내 컴퓨터에 설치해서 사용할 수 있어서 추가 비용이 들지 않으며 인터넷에 연결하지 않은 컴퓨터에서도 보안 문제 없이 언어 모델의 기능을 그대로 활용할 수 있다는 장점이 있습니다.

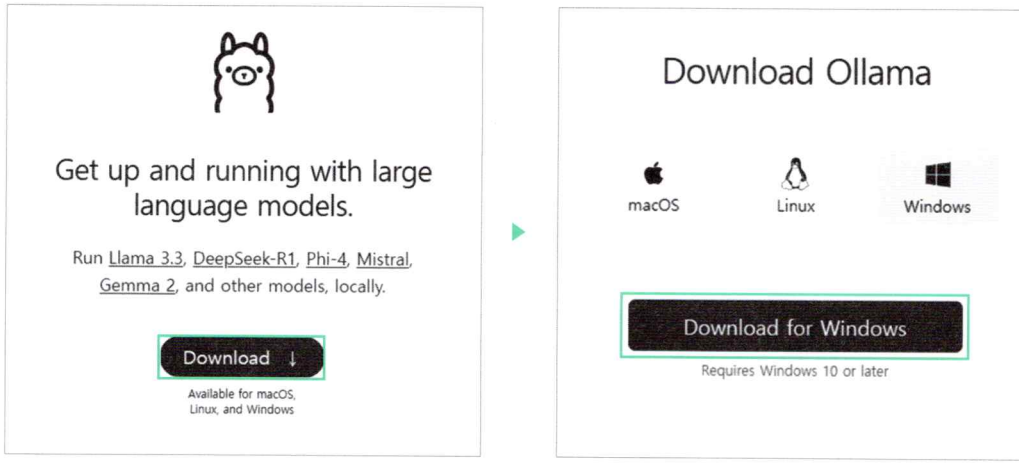
오픈AI가 GPT 모델을 이용해 챗GPT 웹 사이트를 운영하고 API를 서비스하는 것처럼, 딥시크 모델도 챗GPT와 같이 채팅할 수 있는 웹 사이트와 API 서비스를 제공합니다. 그러나 딥시크의 채팅 웹 사이트에서 사용자의 정보를 지나치게 수집하는 것으로 알려졌습니다. 2025년 4월 현재 이 웹 사이트에 접근할 수 없고 많은 국가와 기관에서 딥시크 사이트 접속을 금지하고 있습니다.
이번 장에서 실습하는 내용은 문제가 있던 웹 사이트나 API를 이용하는 것이 아니라, 오픈소스 모델 자체를 내려받아 사용하는 방법입니다. 이 방법을 활용하면 API 비용 부담 없이 내 컴퓨터 안에서 외부에 자료를 전송하지 않고도 딥시크 언어 모델을 사용하는 시스템을 구축할 수 있습니다.

Do it! 실습 올라마와 딥시크-R1 모델 설치하기

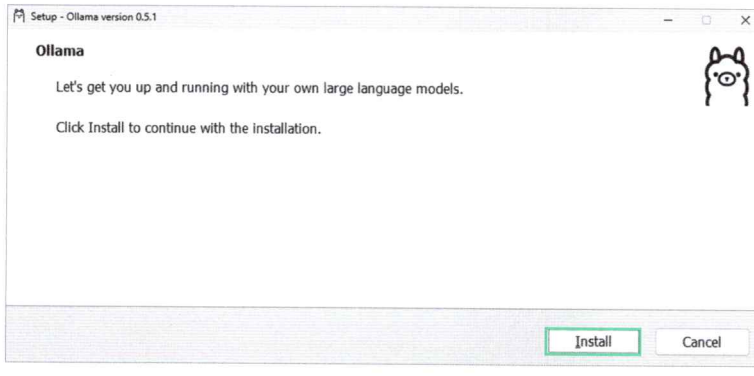
언어 모델을 내 PC에 설치하는 방법은 여러 가지가 있습니다. 허깅페이스에서 언어 모델을 내려받아 사용할 수도 있고, 메타에서 제공하는 올라마Ollama를 이용해서 내려받을 수도 있습니다. 올라마는 인공지능 모델을 효율적으로 배포·실행하는 오픈소스 프레임워크로, 개발자가 쉽고 빠르게 인공지능 모델을 활용할 수 있도록 돕는 도구입니다. 이 플랫폼은 특히 로컬 환경에서 언어 모델을 쉽게 설치하고 실행하는 데 최적화되어 있습니다. 여기에서는 윈도우 운영 체제를 기본으로 올라마를 활용해 딥시크-R1 모델을 내 PC에 설치해 보겠습니다.

✦ 허깅페이스는 05-2절, 05-3절에서 살펴보았습니다.

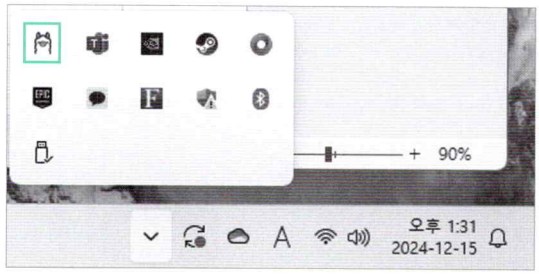
1. 올라마 웹 사이트(https://ollama.com)에 접속하고 [Download] 버튼을 클릭합니다. 그리고 내 컴퓨터의 운영체제에 맞는 설치 프로그램을 내려받으세요.



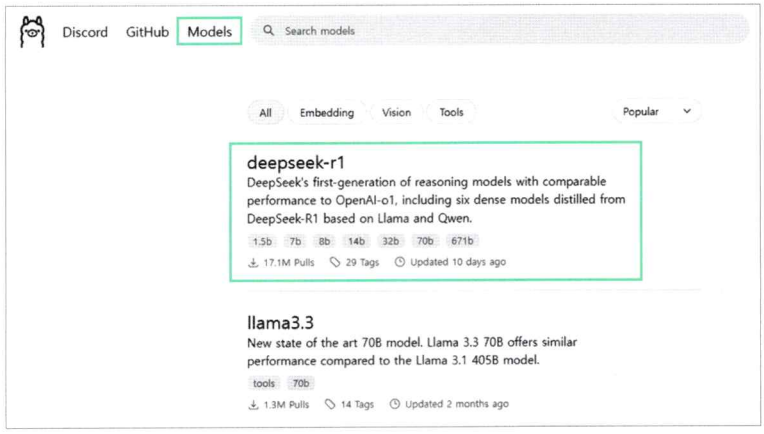
2. 내려받은 프로그램을 실행하고 [Install] 버튼을 클릭해 올라마를 설치합니다.



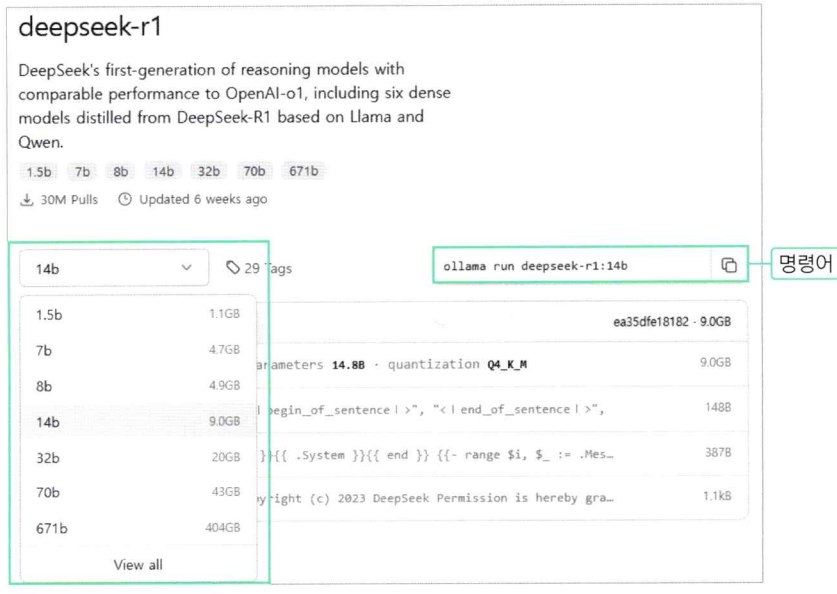
설치가 완료되면 윈도우의 경우 오른쪽 하단에 올라마 아이콘이 생성됩니다.



3. 올라마 웹 사이트에서 [Models]를 클릭하고 [deepseek-r1] 모델을 선택합니다.



4. 딥시크-R1 모델은 경량화 정도에 따라 여러 모델을 제공합니다. 드롭다운 메뉴에서 자신의 컴퓨터 성능에 따라 선택하세요. 저는 14b 모델을 선택했습니다. 모델을 선택하면 오른쪽에 모델을 설치하는 명령어가 생성됩니다.



5. 명령어를 복사해 VS Code 터미널 창에 입력합니다. 이 모델을 처음 사용한다면 설치하는 데 시간이 걸립니다.

```
(venv) > ollama run deepseek-r1:14b
```

만약 다음과 같은 오류가 발생한다면 컴퓨터를 다시 부팅한 후 설치하세요.

```
> ollama run deepseek-r1:14b
ollama : 'ollama' 용어가 cmdlet, 함수, 스크립트 파일 또는 실행할 수 있는 프로그램 이름으로 인식
되지 않습니다. 이름이 정확한지 확인하고 경로가 포함된 경우 경로가 올바른지 검증한 다음 다시 시도하십
시오.
위치 줄:1 문자:1
+ ollama run deepseek-r1:14b
+ ~~~~~~
    + CategoryInfo          : ObjectNotFound: (ollama:String) [], CommandNotFoundEx-
ception
```

설치가 완료되면 다음과 같은 프롬프트가 나옵니다.

```
writing manifest
success
>>> Send a message (/? for help)
```

6. 터미널 창에서 대화를 바로 시작할 수 있습니다. 딥시크-R1은 답변하기 전에 추론 과정을 거치는 모델입니다. 그래서 대답하기 전에 <think>와 </think> 태그 안에서 먼저 생각하는 과정을 거치며 그 과정을 사용자에게 보여 줍니다.

```
>>> Hello! Who are you? Nice to meet you. I'm SungYong.
<think>
Okay, the user just said "Hello!" and introduced themselves as SungYong. They also
mentioned they're nice to meet me. So I should respond in a friendly and welcoming
manner.

I need to make sure to acknowledge their name, maybe add an emoji to keep it warm.
Then offer help since that's part of my role.

Let me put that together in a natural way.
</think>

Hello SungYong! 😊 Nice to meet you too! How can I assist you today?

>>> 와 한국어도 하실 수 있나요? 지금부터 한국어로 대화해요.
<think>
Okay, the user just switched to Korean and asked if I can speak it. They also mentioned
they'll be using Korean from now on.
```

```
I should respond in Korean to make them feel comfortable. Let's confirm that I understand
and offer help.

Maybe add a friendly emoji to keep the conversation warm.
</think>

네, 한국어도 할 수 있습니다! 😊 지금부터 함께 한국어로 대화합시다. 뭐든 도와드리겠습니다!

>>> 와 대단하네요. 저는 요새 GPT에 대한 책을 쓰고 있어요. 딥시크에 대한 책으로 바꿔야 할까요?
<think>
Okay, the user is saying I'm great and that they're currently writing a book about
GPT. They're considering switching it to DeepSeek instead.

I need to acknowledge their compliment first. Then, express interest in their project
and provide helpful suggestions.

Maybe suggest combining both topics or creating a comparative analysis between GPT and
DeepSeek.
</think>

감사합니다! 정말 멋지시네요. 책을 쓰고 계시다니 대단하십니다. GPT에 대해 연구하시면서 어떤 부분이 가
장 흥미로웠는지 궁금해요.

딥시크(DeepSeek)라는 주제로 바꾸시려면, GPT와의 차별점과 연계점을 명확히 설정하시는 것이 좋을 것
같습니다. 예를 들어, 딥시크의 기술적 강점이나 비즈니스 모델에 초점을 맞추거나, 두 AI 간의 비교 분석
도 흥미로운 주제가 될 수 있을 것 같네요.

책이 완성되면ぜひ 읽어드리고 싶어요! 😊
```

대화를 종료하려면 Ctrl + D 를 누릅니다. 대화 내용에서 알 수 있듯이 내 컴퓨터에 설치해서 사용하는 경량화된 언어 모델인데도 답변의 품질이 꽤 좋습니다. 하지만 경량화한 모델의 한계로 종종 일본어나 태국어, 아랍어를 섞어서 답변한다는 문제점도 있습니다.

ollama

11-2 랭체인에서 딥시크 모델 사용하기

랭체인을 활용해 개발하면 언어 모델을 간단히 교체할 수 있습니다. 이번 절에서는 랭체인과 딥시크를 활용해 간단한 챗봇을 만들어 보겠습니다.

Do it! 실습 딥시크와 랭체인으로 챗봇 만들기

결과 파일: chap11/sec02/deepseek_simple_chatbot.py

1. chap11 폴더를 만들고 파이썬 파일을 생성한 후 다음처럼 간단한 챗봇을 만드는 코드를 작성합니다. 08-1절에서 챗봇을 만들 때 사용한 langchain_multiturn.py 파일을 가져와 코드를 수정했습니다. 바뀐 내용은 모델을 초기화할 때 `ChatOpenAI` 대신 `ChatOllama`를 사용한 것입니다. `ChatOllama`에 사용할 모델명을 써주면 설정이 모두 끝납니다.

✦ 저는 14b 모델을 선택했지만 만약 컴퓨터 성능이 부족하다면 8b 혹은 7b 모델을 사용하세요.

딥시크와 랭체인으로 간단한 챗봇 구현하기 deepseek_simple_chatbot.py

```
# from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# 모델 초기화
# llm = ChatOpenAI(model="gpt-4o-mini")
llm = ChatOllama(model="deepseek-r1:14b")  # 만약 컴퓨터 성능이 부족하다면 7b, 8b 모델 선택

messages = [
    SystemMessage("너는 사용자의 질문에 한국어로 답변해야 한다."),
]

while True:
    user_input = input("You\t: ").strip()

    if user_input in ["exit", "quit", "q"]:
        print("Goodbye!")
        break

    messages.append(HumanMessage(user_input))
```

```
    response = llm.invoke(messages)
    print("Bot\t: ", response.content)

    messages.append(AIMessage(response.content))
```

2. ChatOllama를 사용하기 위해 터미널 창에서 패키지를 설치합니다.

✦ ChatOllama를 자세히 알고 싶다면 공식 웹 사이트(https://python.langchain.com/docs/integrations/chat/ollama/)를 참고하세요.

```
venv > pip install langchain-ollama
```

이 코드를 실행하면 다음처럼 터미널 창에서 대화할 수 있습니다. 다만 스트림 출력이 되지 않아 결과를 받기까지 시간이 오래 걸립니다.

```
You: 안녕?
<think>
Alright, the user greeted me with "안녕?" which is a casual way of saying hello in Ko-
rean. I should respond politely but in a friendly manner.
I want to make sure I understand them correctly, so I'll ask if there's something spe-
cific they need help with.
Keeping it natural and not too formal seems right for this context.
</think>
안녕? 도와줄 수 있는 것이 있나요?
```

3. 답변이 스트림 방식으로 출력되도록 코드를 수정해 보겠습니다.

딥시크 스트림 출력하기 — deepseek_simple_chatbot.py

```
# from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# 모델 초기화
# llm = ChatOpenAI(model="gpt-4o-mini")
llm = ChatOllama(model="deepseek-r1:14b")

messages = [
    SystemMessage("너는 사용자의 질문에 한국어로 답변해야 한다."),
]
```

```
while True:
    user_input = input("You\t: ").strip()

    if user_input in ["exit", "quit", "q"]:
        print("Goodbye!")
        break

    messages.append(HumanMessage(user_input))

    response = llm.stream(messages)  ①

    ai_message = None
    for chunk in response:
        print(chunk.content, end="")
        if ai_message is None:
            ai_message = chunk
        else:
            ai_message += chunk
    print('')  ②

    message_only = ai_message.content.split("</think>")[1].strip()
    messages.append(AIMessage(message_only))  ③
```

① .invoke로 되어 있던 부분을 .stream으로 수정해서 스트림 출력이 되도록 합니다.

② for 문을 활용해 response로 스트림 출력되는 부분을 받아 터미널 창에 차례차례 출력하도록 합니다. 이때 ai_message 변수에 response로 온 내용을 계속 덧붙이게 합니다.

③ 이 코드에서 특이한 점은 message_only 변수에 ai_message의 내용 중 </think> 태그 이후 부분만 담도록 한 것입니다. 딥시크-R1 모델은 추론에 특화되어 있어서 답변을 즉시 생성하지 않고 먼저 <think>와 </think> 태그 안에서 자신이 해야 할 일을 생각한 후 이에 맞는 답변을 생성합니다. messages에 딥시크 모델이 생각한 과정까지 포함되면 오히려 자연스러운 답변을 생성하지 못할 수도 있으므로 실제로 대답하는 부분만 대화 기록에 담았습니다.

코드를 실행하면 다음과 같이 대화할 수 있습니다.

```
사용자: 마일즈 데이비스에 대해 알려줘.
<think>
Alright, the user asked about Miles Davis. I need to provide a comprehensive overview.
First, I'll start with his early life, mentioning where and when he was born.
Then, move on to his career highlights, like forming the Modern Jazz Sextet in the
late '40s.
I should include his innovative work in bebop and cool jazz, such as his famous album
"Birth of the Cool."
```

```
(... 생략 ...)
</think>

마일즈 데이비스(Miles Davis)는 20세기 중반부터 후반에 걸쳐 활동한 미국의 재즈 트럼펫리스트, 밴드
리더, 및 은퇴 전 악곡작곡가로, 현대 재 즈 역사상 가장 영향력 있는 아티스트 중 한 명으로 평가받는다.
그는 1940년대부터 1990년대까지 다양한 장르와 스타일로의 음악을 만들었으며, 재즈의 발전에 기여한 것
으로 유명하다.

### 주요 정보
- **출생**: 1926년 5월 26일, 미국 테네시州 채터스톡
- **사망**: 1991년 9월 28일, 미국 뉴욕주 팰리즈리지
- **악기**: 트럼펫

### 경력 및 유명한 작품
1. **초기 경력**:
   - 1940년대 중반, 힐튼 토드 밴드와 함께 활동하며 이름을 떨쳤다.
(... 생략 ...)
```

v3-large
OpenAI 서버로
전송 -> 기밀 넘어가
보안문제 해결x
-> 임베딩모델도 로컬로 해야.
qwen3-embedding으로 변경.
1024vector 뱉음.
벡터 크기 다름.
처음 넣을 때와 질문할 때 같은 임베딩 모델을 사용해야 함.(3072 != 1024)



서버에 기밀정보 넣는 보안 문제로 local 씀.

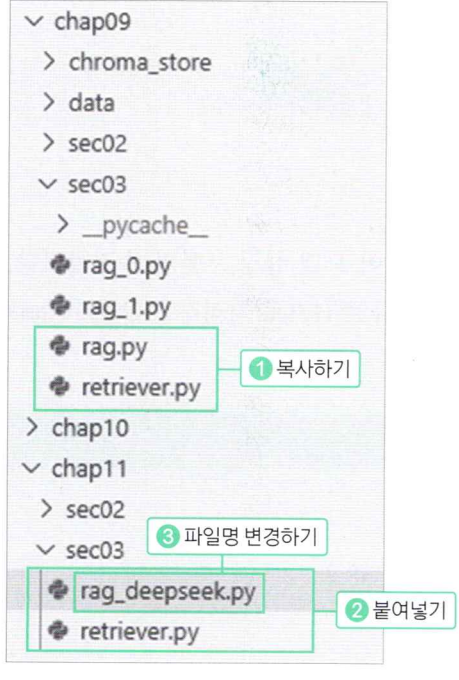
11-3 딥시크에 기반한 RAG 만들기

이번 장에서는 딥시크로 RAG를 만들어 보겠습니다. 09장에서는 스트림릿에서 작동하는 GPT에 기반한 RAG를 만들었습니다. 09장의 최종 코드를 수정하면 간단히 딥시크를 기반으로 하는 RAG로 바꿀 수 있습니다.

Do it! 실습 딥시크로 RAG 만들기

결과 파일: sec03/rag_deepseek.py

1. 09장에서 만들었던 rag.py와 retriever.py라는 파일 2개를 사용합니다. 이 파일들을 chap11에 폴더에 복사한 후, rag.py 파일은 이름을 rag_deepseek.py로 변경합니다.



2. 이 2개 파일에서 GPT로 설정한 언어 모델을 딥시크로 수정합니다. 우선 rag_deepseek.py 파일을 봅시다. `ChatOpenAI`를 사용하는 코드는 주석 처리하고 `ChatOllama`를 활용하도록 수정합니다. `st.title` 부분은 수정하지 않아도 잘 작동하지만 GPT가 아닌 딥시크를 사용한다는 것을 알려 주기 위해 변경합니다.

rag_deepseek.py 파일에서 언어 모델을 딥시크로 변경하기 — rag_deepseek.py

```
# from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

import streamlit as st
import retriever

# 모델 초기화
# llm = ChatOpenAI(model="gpt-4o-mini")
llm = ChatOllama(model="deepseek-r1:14b")
(... 생략 ...)

# Streamlit 앱
st.title("💬 DeepSeek-R1 Langchain Chat")  # 언어 모델을 알려 주기 위해 수정
(... 생략 ...)
```

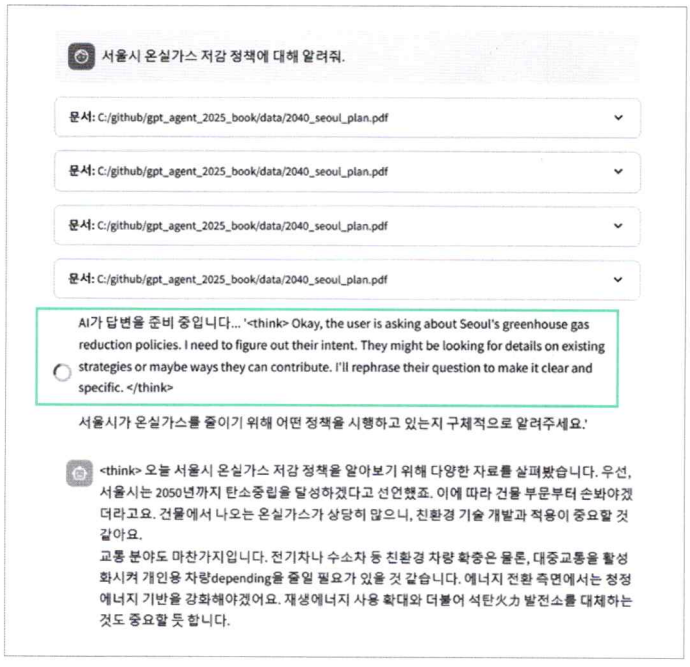
3. chap11 폴더에 복사해 넣은 retriever.py 파일도 언어 모델 설정 부분만 수정합니다. `ChatOpenAI`로 GPT를 사용하도록 설정한 부분을 지우거나 주석으로 처리하고 `ChatOllama`를 사용해 딥시크 모델을 사용하도록 수정하면 끝입니다.

retriever.py 파일에서 언어 모델을 딥시크로 변경하기 — retriever.py

```
# 임베딩 모델 선언하기
from langchain_openai import OpenAIEmbeddings
embedding = OpenAIEmbeddings(model='text-embedding-3-large')

# 언어 모델 불러오기
# from langchain_openai import ChatOpenAI
# llm = ChatOpenAI(model="gpt-4o")
from langchain_ollama import ChatOllama
llm = ChatOllama(model="deepseek-r1:14b")
```

이제 터미널 창에서 `streamlit run` *파일명*`.py`을 입력해 실행해 봅시다. 딥시크 모델 특성상 `<think>`와 `</think>` 태그 안에 무슨 생각을 하고 답변을 생성할지 보여 주는 과정이 출력됩니다. 이 부분을 드러나지 않도록 처리할 수도 있지만 사용자가 알아 두면 재미있고 유용한 정보라서 남겨 두었습니다.



딥시크처럼 로컬에 설치해 사용할 수 있는 언어 모델을 이용하면 토큰 비용을 지출할 필요가 없고 보안 문서도 아무 걱정 없이 활용할 수 있다는 장점이 있습니다. 다만 이 예제에서는 이미 임베딩되어 있는 벡터 DB를 대상으로 RAG 대화를 했습니다. 09장에서 문서를 임베딩할 때는 오픈AI의 임베딩 API를 사용했으므로 그 과정에서 우리가 사용한 PDF 파일 내용이 오픈AI에 전송됩니다. 이 임베딩 과정도 임베딩 모델을 로컬에 설치해서 사용할 수 있습니다. 임베딩 모델과 언어 모델 모두 로컬에 설치해 사용한다면 내부 문서도 노출될 걱정 없이 사용할 수 있습니다.

✦ 임베딩 모델을 로컬에 설치하는 방법은 16-1절에서 다룹니다.

한 걸음 더! 랭체인으로 만든 모든 애플리케이션을 GPT가 아닌 딥시크-R1 모델로 바꿀 수 있나요?

아직 모든 애플리케이션을 바꿀 수는 없습니다. 딥시크-R1은 등장한 지 얼마 되지 않은 모델이라 몇 가지 제약이 있습니다. 예를 들어 10장에서는 도구 호출을 이용하는 기능을 활용했는데 딥시크-R1 모델은 도구가 호출되지 않습니다. 이런 상황에서 언어 모델을 딥시크-R1으로 변경하면 다음처럼 오류가 발생합니다.

```
ollama._types.ResponseError: registry.ollama.ai/library/deepseek-r1:14b does not
support tools (status code: 400)
```

이런 경우에는 도구 호출을 지원하는 llama3.2:3b와 같은 다른 모델을 선택해야 합니다. 그러나 llama3.2:3b 모델은 한국어가 일본어, 중국어, 영어, 태국어와 섞여서 출력되는 한계가 있습니다. 이 책을 읽는 시점에는 더 나은 모델이 공개되었을 수 있으므로 적절한 모델을 선택해 활용하길 바랍니다.